



Marwadi
University
Marwadi Chandarana Group

NAAC



MARWADI UNIVERSITY

FACULTY OF COMPUTER APPLICATIONS

Programming with Python & Problem Solving (05MC0107)

LAB MANUAL

Programming with Python & Problem Solving (05MC0107) Lab Manual (2026-27)

Program: MCA

Semester: 1

Name of the Student: Kundariya Vasu Rameshbhai

Enrolment No. :92600584063

I

NDEX

Lab	Program(unit 2)	Date	Remarks	Signature
1	Write a program to demonstrate conditional statements using if if-else and if-elif-else..			
2	Write a program to check whether a number is positive negative or zero using nested conditions.			
3	Write a program to generate a multiplication table using a for loop.			
4	Write a program to find the sum of digits of a number using a while loop.			
5	Write a program to demonstrate the use of break continue and pass statements.			
6	Write a program to iterate over lists strings and dictionaries using loops.			
7	Write a program to demonstrate a list dictionary and set comprehensions.			
8	Write a program to explain mutable and immutable objects in Python.			
9	Write a program to demonstrate iterators and iterables in Python.			
10	10. Write a program to generate a sequence of numbers using generator functions and yield keyword.			

Practical No. 1

Definition : Write a program to demonstrate conditional statements using if if-else and if-elif-else.

Aim/ Objective:

- To understand the use of conditional statements in Python.
- To learn how to use if, if-else, and if-elif-else statements.
- To make decisions based on given conditions.
- To understand how conditions control the flow of a Python program.

Source Code:

```
# if Statment
```

```
age = int(input("Enter your age: "))
```

```
if age>18:
```

```
    print("You are eligible to vote.")
```

```
# if else statment
```

```
num = int(input("Enter any nuber = "))
```

```
if num %2==0:
```

```
    print("Number is Even ")
```

```
else:
```

```
    print("Number is odd ")
```

```
# if-elif-else.
```

```
marks =int(input("Enter any markas ="))
```

```
if marks>90:
```

```
    print("Grade A ")
```

```
elif marks>75:
```

```
    print("Grade B")
```

```
elif marks>50:  
    print("Grade C")  
elif marks>33:  
    print("Grade D")  
else:  
    print("Fail")
```

Output:

```
Enter your age: 22  
You are eligible to vote.  
Enter any nuber = 4  
Number is Even  
Enter any markas =65  
Grade C
```

Conclusion:

The program successfully demonstrates the use of conditional statements in Python. It shows how if, if-else, and if-elif-else statements are used to check conditions and make decisions. Thus, the basic concept of decision-making and control flow in Python is understood.

Practical No. 2

Definition : Write a program to check whether a number is negative or zero using nested conditions

Aim/ Objective:

- To understand the concept of nested if statements in Python.
- To check whether a given number is negative or zero.
- To practice decision-making using conditional statements.

Source Code:

```
num = int(input("Enter any Number :"))  
  
if num<=0:  
    if num==0:  
        print("Number is zero")  
    else:  
        print("Number is Nagativ")  
else :  
    print("Number is Possitive")
```

Output:

Enter any Number :4

Number is Possitive

Enter any Number :0

Number is zero

Enter any Number :-2

Number is Nagativ

Conclusion:

The program successfully demonstrates the use of **nested conditional statements** in Python. It checks whether the entered number is negative, zero, or positive by using an if statement inside another if statement.

Practical No. 3

Definition : Write a program to generate a multiplication table using a for loop.

Aim/ Objective:

- To understand the use of the for loop in Python.
- To generate a multiplication table for a given number.
- To practice iteration and arithmetic operations in Python.

Source Code:

```
num = int(input("Enter Any Number"))  
  
for i in range(1,11):  
  
    print(num,"x",i,"=",num*i)
```

Output:

Enter Any Number5

5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50"

Conclusion:

The program successfully demonstrates the use of a for loop in Python to generate the multiplication table of a given number. It shows how loops can be used to execute repetitive operations efficiently.

Practical No. 4

Definition : Write a program to find the sum of digits of a number using a while loop.

Aim/ Objective:

- To understand the use of the while loop in Python.
- To find the sum of all digits of a given number.
- To practice arithmetic operations and looping in Python.

Source Code:

```
num= int(input("Enter any number :"))  
  
sum =0  
  
while num>0:  
  
    digit =num % 10  
  
    sum= sum+ digit  
  
    num= num//10  
  
print("Sum is a =",sum)
```

Output:

Enter any number :1234567

Sum is a = 28

Conclusion:

The program successfully demonstrates the use of a while loop in Python to find the sum of the digits of a given number. It shows how the % operator is used to extract digits and the // operator is used to remove digits from the number.

Practical No. 5

Definition : Write a program to demonstrate the use of break continue and pass statements.

Aim/ Objective:

- To understand the use of break, continue, and pass statements in Python.
- To learn how these statements control the flow of a loop.
- To understand how to terminate, skip, or leave a block of code empty.

Source Code:

```
print("-----Break-----")
```

```
for i in range(1,11):
```

```
    if i ==4:
```

```
        break
```

```
    print(i)
```

```
print("-----Continue-----")
```

```
for i in range(1,11):
```

```
    if i ==4:
```

```
        continue
```

```
    print(i)
```

```
print("-----Pass-----")
```

```
for i in range(1,11):
```

```
    if i ==4:
```

```
        pass
```

```
    print(i)
```

Output:

```
-----Break-----
```

```
1
```

```
2
```

3

-----Continue-----

1

2

3

5

6

7

8

9

10

-----Pass-----

1

2

3

4

5

6

7

8

9

10

Conclusion:

The program successfully demonstrates the use of break, continue, and pass statements in Python. The break statement terminates the loop, continue skips the current iteration, and pass allows the program to continue without performing any operation.

Practical No. 6

Definition : . Write a program to iterate over lists strings and dictionaries using loops.

Aim/ Objective:

- To understand how loops are used to iterate over different data types in Python.
- To iterate through a list, string, and dictionary using loops.
- To understand how to access elements and values from different data structures.

Source Code:

```
num =[10,20,30,40,50]
```

```
print(num)
```

```
for i in num:
```

```
    print(i)
```

```
name ="Vasu"
```

```
print(name)
```

```
for ch in name:
```

```
    print(ch)
```

```
student = {
```

```
    "name":"vasu",
```

```
    "age":21,
```

```
    "Course":"Python"
```

```
}
```

```
for key,value in student.items():
```

```
    print(key,":",value)
```

Output:

```
num =[10,20,30,40,50]
```

```
print(num)
```

```
for i in num:
```

```
    print(i)
```

```
name ="Vasu"
```

```
print(name)
```

```
for ch in name:
```

```
    print(ch)
```

```
student = {
```

```
    "name":"vasu",
```

```
    "age":21,
```

```
    "Course":"Python"
```

```
}
```

```
for key,value in student.items():
```

```
    print(key,":",value)
```

Conclusion:

The program successfully demonstrates how to iterate over lists, strings, and dictionaries using loops in Python. It shows how the for loop can be used to access each element of a list, each character of a string, and each key-value pair of a dictionary.

Practical No. 7

Definition : Write a program to demonstrate list dictionary and set comprehensions.

Aim/ Objective:

- To understand the concept of comprehensions in Python.
- To create lists, dictionaries, and sets using comprehensions.
- To perform operations on collections in a simple and efficient way.

Source Code:

```
num = [1,2,3,4,5]

print(num)

sq =[x*x for x in num]

print("Square =",sq)

sq_digit ={x:x*x for x in num}

print("Square Digit :",sq_digit)

even_num={x for x in num if x % 2==0}

print("Even number : ",even_num)
```

Output:

```
[1, 2, 3, 4, 5]

Square = [1, 4, 9, 16, 25]

Square Digit : {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Even number : {2, 4}
```

Conclusion:

The program successfully demonstrates **list, dictionary, and set comprehensions** in Python. It shows how comprehensions can be used to create and process collections in a concise and efficient manner.

Practical No. 8

Definition : Write a program to illustrate variable scope using local global and nonlocal variables.

Aim/ Objective:

- To understand the concept of variable scope in Python.
- To demonstrate local, global, and nonlocal variables.
- To understand where variables can be accessed and modified in a program.

Source Code:

```
x=10

def outer():
    y=20
    def inner():
        nonlocal y
        y=30
        print("Global variable :",x)
        print("Nonlocal variable :",y)
    inner()
    print("local variable : ",y)
outer()
```

Output:

Global variable : 10

Nonlocal variable : 30

local variable : 30

Conclusion:

The program successfully demonstrates local, global, and nonlocal variable scopes in Python. It shows how the scope of a variable determines where it can be accessed and modified within a program.

Practical No. 9

Definition : Write a program to demonstrate iterators and iterables in Python.

Aim/ Objective:

- To understand the concept of **iterables and iterators** in Python.
- To learn how to create an iterator using the `iter()` function.
- To access elements of an iterator using the `next()` function.

Source Code:

```
num = [10,20,30,40,50,60]

print(num)

print("Iterable:")

for i in num:

    print (i)

print(" Iterators:")

num_iter = iter(num)

print(next(num_iter))

print(next(num_iter))

print(next(num_iter))

print(next(num_iter))

print(next(num_iter))

print(next(num_iter))

print(next(num_iter))
```

Output:

```
[10, 20, 30, 40, 50, 60]

Iterable:

10

20

30
```

40

50

60

Iterators:

10

20

30

40

50

60

Conclusion:

The program successfully demonstrates iterables and iterators in Python. It shows how an iterable can be converted into an iterator using `iter()` and how elements can be accessed one by one using `next()`.

Practical No. 10

Definition : Write a program to generate a sequence of numbers using generator functions and yield keyword.

Aim/ Objective:

- To understand the concept of **generator functions** in Python.
- To learn the use of the **yield** keyword.
- To generate a sequence of numbers one at a time.
- To understand how generators save memory by producing values when required
-

Source Code:

```
def num_gen(n):  
    for i in range(1,1+n):  
        yield i  
  
n=int(input("Enter Any number ="))  
  
print("Number Of Sequence ")  
for num in num_gen(n):  
    print(num)
```

Output:

Enter Any number =6

Number Of Sequence

1

2

3

4

5

6

Conclusion:

The program successfully demonstrates the use of generator functions and the yield keyword in Python. It generates a sequence of numbers one at a time, which makes generators useful for handling large sequences efficiently.